

Backend Engineering — Day 1

1. What is Backend Engineering?

Backend engineering is **much more than building CRUD APIs**.

A good backend system should be:

- **Reliable** — works correctly and consistently.
- **Scalable** — can handle increasing users, traffic, and data.
- **Fault-tolerant** — can continue functioning even when some components fail.
- **Maintainable** — easy for developers to understand, modify, and extend.
- **Efficient** — uses CPU, memory, network, and database resources effectively.

Important mindset

Do not learn backend development only through a particular framework such as:

- Express
- Spring Boot
- Ruby on Rails
- etc.

If you only understand the framework, your knowledge becomes tied to that ecosystem.

Instead, understand the **underlying concepts**:

HTTP → Networking → Routing → Serialization → Authentication → Validation → Middleware
→ Business Logic → Database → Caching → Queues → Observability → Security → Scaling →
Deployment

Then switching from one language/framework to another becomes much easier.

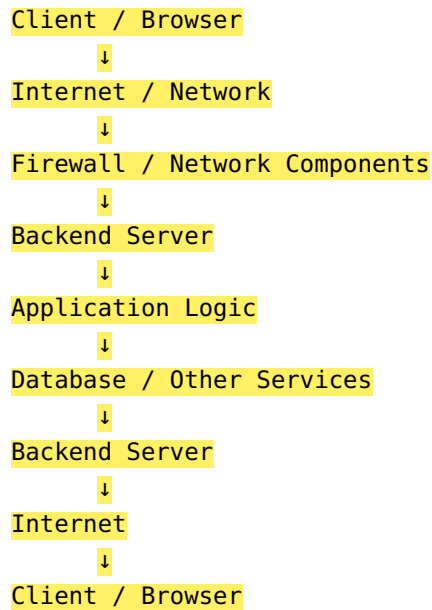
For example:

If you understand how authentication, HTTP, databases, caching, queues, and concurrency work internally, moving from Express to Go or Spring Boot is primarily learning a new implementation rather than relearning backend engineering itself.

2. High-Level Understanding of a Backend System

Before learning individual backend components, understand the overall flow.

A typical request looks approximately like:



Example

Suppose you visit:

`https://example.com/users/123`

The browser sends a request.

The request travels through the network and eventually reaches the backend server.

The backend:

1. Receives the request.
2. Determines which route should handle it.
3. Runs middleware.
4. Validates the request.
5. Checks authentication/authorization if required.
6. Executes business logic.
7. Interacts with the database if necessary.
8. Creates a response.
9. Sends the response back to the client.

Understanding this complete flow is important because backend engineering is essentially about building and operating this entire system reliably.

3. HTTP Protocol

HTTP (**HyperText Transfer Protocol**) is one of the fundamental protocols used for communication between clients and servers.

It defines how requests and responses are exchanged.

Client — HTTP Request —→ Server

Client ← HTTP Response — Server

3.1 HTTP Request

An HTTP request contains information such as:

- HTTP method
- URL/path
- Headers
- Request body

Example:

```
POST /users HTTP/1.1
Host: example.com
Content-Type: application/json

{
  "name": "Sourabh",
  "age": 20
}
```

3.2 HTTP Headers

Headers provide additional information about a request or response.

Important categories include:

Request Headers

Information sent by the client.

Examples:

Authorization
Content-Type
Accept
User-Agent
Cookie

Representation Headers

Describe the representation of the data being transferred.

Examples include information about:

Content-Type
Content-Encoding
Content-Length

General Headers

Headers that provide general information about the message/connection.

Security Headers

Headers used to improve application security.

Examples include:

Strict-Transport-Security
Content-Security-Policy
X-Content-Type-Options

4. HTTP Methods

HTTP methods describe what operation the client wants to perform.

Common methods:

Method	Typical Purpose
GET	Retrieve data
POST	Create/submit data

Method	Typical Purpose
PUT	Replace/update a resource
PATCH	Partially update a resource
DELETE	Delete a resource

Example:

```
GET /users
```

Fetch users.

```
POST /users
```

Create a user.

```
PUT /users/10
```

Replace/update user 10.

```
DELETE /users/10
```

Delete user 10.

Important

HTTP methods have **semantics**. Don't randomly use POST for everything.

A well-designed API uses methods according to their intended meaning.

5. CORS

CORS stands for:

Cross-Origin Resource Sharing

It controls whether a browser is allowed to make requests between different origins.

For example:

Frontend:
https://frontend.com

Backend:
https://api.backend.com

These are different origins.

The browser applies security restrictions to such requests.

5.1 Simple Request vs Preflight Request

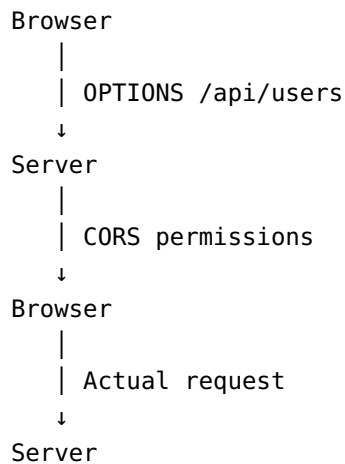
A **simple request** can be sent directly if it satisfies the conditions for a CORS simple request.

For some cross-origin requests, the browser first sends a **preflight request**.

The preflight request uses:

OPTIONS

Example flow:



The server indicates which origins, methods, or headers are allowed.

6. HTTP Responses

A response generally contains:

```
Status Line  
Headers  
Body
```

Example:

```
HTTP/1.1 200 OK  
Content-Type: application/json  
  
{  
  "name": "Sourabh"  
}
```

6.1 HTTP Status Codes

Status codes tell the client what happened.

2xx — Success

```
200 OK  
201 Created
```

4xx — Client-side problems

```
400 Bad Request  
401 Unauthorized  
403 Forbidden  
404 Not Found
```

5xx — Server-side problems

```
500 Internal Server Error
```

Important distinction

401 → authentication is missing/invalid

403 → authenticated but not allowed

7. HTTP Caching

Caching allows frequently requested data to be reused instead of generating it again.

This can reduce:

- Server processing
- Database queries
- Network usage
- Response time

HTTP caching uses headers such as:

ETag
Cache-Control
Max-Age

ETag

An ETag identifies a particular version of a resource.

The client can later tell the server:

"I already have this version. Has it changed?"

If nothing changed, the server can avoid sending the entire resource again.

Max-Age

Specifies how long a response can be considered fresh.

8. HTTP Versions

Important HTTP versions:

HTTP/1.1
HTTP/2
HTTP/3

The major evolution includes improvements in:

- Performance
- Connection handling
- Multiplexing
- Network efficiency
- Transport mechanisms

HTTP/2 improves handling of multiple streams over a connection.

HTTP/3 uses QUIC rather than TCP as its transport protocol.

The important goal is:

Understand why newer HTTP versions exist and what problems they solve.

9. Content Negotiation

Content negotiation allows the client and server to agree on an appropriate representation of data.

Headers can communicate preferences such as:

Accept
Accept-Encoding

For example, a client may indicate that it accepts:

application/json

The server can then respond using an appropriate representation.

10. Persistent Connections

Persistent connections allow multiple HTTP requests/responses to reuse a connection instead of establishing a completely new connection for every request.

This reduces connection overhead and improves performance.

11. HTTP Compression

Compression reduces the size of data transferred over the network.

Common compression techniques mentioned are:

```
gzip
deflate
Brotli (br)
```

Smaller payloads generally mean:

```
Less data transferred
    ↓
Less network overhead
    ↓
Faster responses
```

12. SSL, TLS and HTTPS

HTTP

Communication is not inherently encrypted.

HTTPS

HTTPS = HTTP over TLS.

TLS provides encryption and helps provide:

- Confidentiality
- Integrity
- Authentication of the server

The important idea is:

```
HTTP
↓
```

TLS encryption
↓
HTTPS

13. Routing

Routing maps incoming URLs to server-side logic.

Example:

GET /users
↓
getUsers()

POST /users
↓
createUser()

Routing connects:

HTTP Method + URL
↓
Route
↓
Handler / Controller

13.1 Components of Routes

Path Parameters

Used to identify a specific resource.

/users/:id

Example:

```
/users/123
```

Here:

```
id = 123
```

Query Parameters

Used to provide additional options to a request.

```
/users?page=2&limit=10
```

Here:

```
page = 2  
limit = 10
```

14. Types of Routes

Static Route

```
/users
```

Dynamic Route

```
/users/:id
```

Nested Route

```
/users/:userId/orders
```

Hierarchical Route

Represents resources in a hierarchy.

Catch-all / Wildcard Route

Handles multiple unmatched paths.

Regular Expression-based Route

Routes can also use patterns to match URLs.

15. API Versioning

APIs change over time.

Instead of suddenly breaking existing clients, APIs can be versioned.

Example:

```
/api/v1/users  
/api/v2/users
```

Versioning allows old and new clients to coexist.

Other approaches include:

- URI versioning
 - Header versioning
 - Query-string versioning
 - Media-type versioning
-

API Deprecation

When an old API version is no longer preferred:

```
v1 → deprecated  
v2 → recommended
```

The old version should generally be phased out carefully instead of immediately removing it.

16. Route Grouping

Routes can be grouped to share:

- Middleware
- Permissions
- Versioning
- Common configuration

Example:

```
/api/v1
├── /users
├── /orders
└── /payments
```

This makes large applications easier to organize.

17. Serialization and Deserialization

Data inside a program exists in its native data structures.

Before sending it across a network, it must usually be converted into a transferable format.

This is **serialization**.

```
Native Object
  ↓
Serialization
  ↓
JSON / XML / Protobuf
  ↓
Network
```

When the receiver converts the transferred representation back into its native structure, this is **deserialization**.

```
JSON / XML / Protobuf
  ↓
Deserialization
```



Native Object

18. Serialization Formats

Two broad categories:

Text-based

Examples:

JSON

XML

Advantages:

- Human-readable
- Easy to inspect/debug
- Widely supported

Binary

Example:

Protocol Buffers (Protobuf)

Advantages:

- More compact
- Generally faster to process/transfer
- Useful when performance matters

Trade-off:

Text → readability
Binary → efficiency

19. JSON

JSON is one of the most commonly used formats for APIs.

Example:

```
{  
  "name": "Sourabh",  
  "age": 20,  
  "student": true,  
  "skills": ["C++", "JavaScript"]  
}
```

JSON supports:

- Strings
 - Numbers
 - Booleans
 - Arrays
 - Objects
 - Null
-

19.1 Nested JSON

Objects and arrays can be nested.

```
{  
  "user": {  
    "name": "Sourabh",  
    "skills": ["C++", "JavaScript"]  
  }  
}
```

20. JSON Deserialization

Different programming languages convert JSON into their native structures.

Examples:

```
Python → dictionary  
JavaScript → object  
Go → struct
```

21. Common JSON Problems

Be careful with:

Missing fields

```
{  
  "name": "Sourabh"  
}
```

What happens if `age` is required?

Extra fields

The client may send fields that the server does not expect.

Null values

```
{  
  "age": null  
}
```

The application must handle them correctly.

Dates

Date formats can create problems.

Time zones

Different systems may represent timestamps differently.

Use a consistent representation.

22. Custom Serialization

Sometimes the default serialization behavior isn't enough.

You may need to:

- Change field names
- Transform values
- Hide sensitive fields
- Format dates
- Convert custom objects

23. Serialization Errors & Security

Possible problems include:

- Invalid data
- Conversion failures
- Unknown fields
- Invalid JSON
- Unexpected data types

Validation should happen before processing data.

JSON schemas can be used to validate whether incoming data follows an expected structure.

24. Serialization Performance

Performance can be improved by:

- Removing unnecessary fields
- Compressing payloads
- Choosing appropriate formats

Example:

JSON

↓

Readable but potentially larger

Protobuf

↓

Compact and efficient but less human-readable

The correct choice depends on the application's requirements.

25. Authentication and Authorization

These are different concepts.

Authentication

Answers:

Who are you?

Example:

Login with username/password

Authorization

Answers:

What are you allowed to do?

Example:

Admin → delete users
Normal user → cannot delete users

Flow:

Authentication
↓
Identify user
↓
Authorization
↓
Check permissions

26. Authentication Types

The transcript covers:

- Stateful authentication
 - Stateless/token authentication
 - Basic authentication
 - Sessions
 - JWT/JWS
 - Cookies
 - OAuth
 - OpenID Connect
 - API keys
 - Multi-factor authentication
-

27. Sessions

A session stores authentication state associated with a user.

Conceptually:

```
graph TD; User --> Login; Login --> SessionCreated[Session created]; SessionCreated --> SessionID[Session ID]; SessionID --> ClientSends[Client sends session ID]; ClientSends --> ServerIdentifies[Server identifies session];
```

28. JWT / JWS

Token-based authentication can use signed tokens.

A JWT commonly contains:

```
Header
Payload
Signature
```

The signature helps verify that the token has not been modified.

Important:

A signed token is not automatically encrypted.

Do not put sensitive secrets into a normal JWT payload assuming it is hidden.

29. Cookies

Cookies can be used to store information such as session identifiers or authentication tokens.

Security-related cookie properties include concepts such as:

```
HttpOnly
Secure
SameSite
```

These help reduce certain classes of attacks.

30. OAuth and OpenID Connect

OAuth

Primarily deals with delegated authorization.

It allows an application to obtain limited access to resources without directly receiving the user's password.

OpenID Connect

Built on OAuth and adds an identity layer.

It is used for authentication/identity.

31. API Keys

API keys are credentials commonly used to identify or authorize API clients.

They are frequently used for:

- Service-to-service communication
- Third-party APIs
- Developer APIs

They must be protected like credentials.

32. Multi-Factor Authentication

MFA requires multiple forms of verification.

For example:

Password
+
OTP

Even if one factor is compromised, another factor provides additional protection.

33. Password Security

Two important concepts:

Hashing

Passwords should not be stored as plain text.

Instead:

Password
↓
Hashing algorithm
↓
Password hash

During login, the supplied password is checked against the stored hash.

Salting

A salt is additional random data used during password hashing.

It helps protect against attacks involving precomputed password hashes.

34. Authorization Models

Common models include:

RBAC

Role-Based Access Control

Permissions are assigned to roles.

Example:

```
Admin → delete user  
Editor → edit content  
User → view content
```

ABAC

Attribute-Based Access Control

Access depends on attributes such as:

- User
- Resource
- Environment
- Context

The transcript also mentions RBAC/related authorization approaches as part of authorization design.

35. Authentication & Authorization Security

Important practices:

- Secure cookies
 - Prevent CSRF
 - Prevent XSS
 - Protect against MITM attacks
 - Rate-limit authentication attempts
 - Implement account lockout where appropriate
 - Audit authentication events
 - Avoid leaking sensitive information
 - Prevent privilege escalation
-

36. Avoid Information Leakage

Bad:

```
"Username exists but password is wrong."
```

This tells an attacker that the username exists.

Better:

```
"Invalid credentials."
```

The system should avoid giving attackers unnecessary information.

37. Timing Attacks

Attackers can sometimes use differences in response time to infer information.

For example:

```
Invalid username → quick response
```

```
Valid username + password verification → slower response
```

An attacker could potentially use many measurements to infer whether a username exists.

Therefore, authentication systems should avoid unnecessary timing differences that leak sensitive information.

38. Validation and Transformation

Validation checks whether incoming data is acceptable.

Transformation converts incoming data into the required format.

39. Types of Validation

39.1 Syntactic Validation

Checks whether data follows the correct format.

Examples:

```
Is this a valid email?  
Is this a valid phone number?  
Is this a valid date format?
```

39.2 Semantic Validation

Checks whether the value makes logical sense.

Example:

```
Date of birth cannot be in the future.
```

Another example:

```
Age must be between 1 and 120.
```

39.3 Type Validation

Checks whether data has the expected type.

Example:

```
name → string  
age → integer  
skills → array  
user → object
```

40. Client-Side vs Server-Side Validation

Client-side validation

Advantages:

- Immediate feedback
- Better user experience
- Reduces unnecessary requests

But:

Client-side validation is NOT a security boundary.

The client can be modified or bypassed.

Server-side validation

The server must validate data before using it in business logic.

Why?

Because the server is the gateway to:

- Business logic
- Database
- Sensitive operations

Therefore:

Client validation → UX
Server validation → Security + correctness

You need both.

41. Failing Fast

Fail early when invalid input is detected.

Example:

```
Request
↓
Validate
↓
Invalid?
↓
Return immediately
```

Don't continue performing expensive operations if the request is already invalid.

Benefits:

- Less CPU usage
 - Less database work
 - Lower latency
 - Cleaner logic
-

42. Transformation

Transformation converts data into the format required by the application.

Example:

Query parameter:

```
?id=123
```

HTTP query parameters are received as strings.

Conceptually:

```
"123"  
↓  
123
```

This conversion is a transformation.

43. Date Transformation

Different systems may send dates in different formats.

For example:

```
2026-09-15
```

or

```
2026-09-15T12:30:00Z
```

The validation pipeline may need to normalize these into a consistent internal representation.

44. Normalization

Normalization makes equivalent input consistent.

Examples:

Email

```
SOURABH@EXAMPLE.COM  
↓  
sourabh@example.com
```

Whitespace

```
"  Sourabh  "
      ↓
"Sourabh"
```

Phone number

Convert into a consistent representation, such as adding the country code where appropriate.

45. Sanitization

Sanitization removes or neutralizes potentially dangerous input.

It is important for security.

Examples of attacks that require defensive handling include:

```
SQL injection
XSS
```

However, sanitization should not be treated as a substitute for safe database APIs, parameterized queries, or proper output encoding.

46. Complex Validation

Relationship-based validation

Example:

```
password
confirmPassword
```

Validation:

```
password == confirmPassword
```

Conditional validation

Example:

```
married = true
```

Then:

```
partnerName → required
```

If:

```
married = false
```

then:

```
partnerName → may not be required
```

47. Chain Validation

Multiple transformations/validations can be performed in sequence.

Example:

```
Input
↓
Convert to lowercase
↓
Remove/handle unwanted characters
↓
Check length
↓
Validate
```

The order matters.

48. Validation Error Handling

Errors should be:

- Meaningful
- Consistent
- Useful to the client
- Safe from information leakage

Multiple validation errors can be returned together.

Example:

```
{
  "errors": [
    "Email is invalid",
    "Password must contain at least 8 characters"
  ]
}
```

But authentication errors should avoid revealing sensitive information.

Instead of:

```
Invalid password
```

prefer:

```
Invalid credentials
```

49. Gracefully Handling Transformation Failures

Examples:

```
Invalid JSON
Invalid date
Invalid number
Invalid type conversion
```

These should produce appropriate client-facing errors instead of crashing the application.

50. Middleware

Middleware is logic that executes during the request/response lifecycle.

Conceptually:

```
Request
↓
Middleware 1
↓
Middleware 2
↓
Middleware 3
↓
Handler
↓
Response
```

Middleware can:

- Inspect requests
- Modify requests
- Perform checks
- Stop requests
- Pass control to the next middleware

51. Middleware Chaining

Middleware is commonly executed sequentially.

Example:

```
Request
↓
Logger
↓
Authentication
↓
Validation
```

```
↓  
Route Handler  
↓  
Response
```

A middleware typically passes control forward.

In frameworks such as Express, this is commonly associated with:

```
next()
```

52. Middleware Ordering

Order matters.

A reasonable flow can be:

```
Logging  
↓  
Authentication  
↓  
Validation  
↓  
Route Handling  
↓  
Error Handling
```

Incorrect ordering can cause:

- Security problems
- Unnecessary processing
- Incorrect behavior

53. Common Middleware

Security Middleware

Can add security headers such as:

X-Content-Type-Options
Strict-Transport-Security
Content-Security-Policy

CORS Middleware

Controls cross-origin behavior.

CSRF Middleware

Helps defend against CSRF attacks.

Rate Limiting Middleware

Limits how frequently clients can make requests.

Authentication Middleware

Protects routes requiring authenticated users.

Logging Middleware

Records request information.

Error Handling Middleware

Centralizes application error formatting.

Compression Middleware

Compresses response bodies.

Body Parsing Middleware

Processes incoming:

JSON
URL-encoded forms
Multipart/form-data

Multipart requests are commonly used for file uploads.

54. Middleware Performance

Keep middleware:

- Lightweight
- Efficient
- Properly ordered

Don't put expensive operations into middleware unnecessarily.

Middleware affects many requests, so inefficient middleware can become a system-wide bottleneck.

55. Request Context

Request context is temporary state associated with a particular request.

It exists only during the request lifecycle.

Example:

```
Request
↓
Authentication Middleware
↓
User information added to context
↓
Controller
↓
Service
```

The controller/service can access information associated with that request.

56. What Can Request Context Contain?

Request Metadata

- HTTP method
- URL
- Headers
- Query parameters
- Body

User/Session Information

Authentication middleware may determine the user and attach relevant information to the request context.

Tracking Information

Examples:

```
Request ID
Trace ID
```

These are extremely useful for debugging and tracing requests across systems.

Request-Specific Data

Examples:

- Permission checks
- Cached values
- Temporary data generated during processing

57. Request Context Use Cases

Request context is useful for:

- Authentication
- Authorization
- Rate limiting
- Logging
- Tracing

58. Timeouts and Cancellation

Requests should not necessarily run forever.

Important concepts:

- Request timeouts
- Custom timeouts
- Cancellation signals

Timeouts prevent resources from being occupied indefinitely by slow operations.

59. Request Context Best Practices

Keep context:

- Lightweight
- Request-scoped
- Temporary

Avoid using request context as a global dumping ground for arbitrary application state.

Also ensure request-specific resources are cleaned up to prevent memory/resource leaks.

60. Handlers, Controllers and Services

Backend applications often separate responsibilities.

A common structure is:

```
Route
↓
Controller / Handler
↓
Service
↓
Data Access Layer
↓
Database
```

61. Handler / Controller

The controller/handler deals primarily with the HTTP layer.

Typical responsibilities:

- Receive request
- Extract input
- Call service
- Return response
- Map errors to HTTP responses

It should not contain huge amounts of business logic.

62. Service Layer

The service layer contains business logic.

Example:

```
Controller
  ↓
Booking Service
  ↓
Check car availability
  ↓
Calculate price
  ↓
Create booking
```

The service should represent what the application **does**, rather than how HTTP works.

63. MVC

MVC stands for:

```
Model
View
Controller
```

In backend/API systems, the exact implementation varies, but the broader idea is separation of responsibilities.

The important concept is:

Don't put everything into one giant controller.

64. Centralized Error Handling

Instead of every controller manually formatting every error differently, applications can centralize error handling.

This gives consistent responses.

Example:

```
{
  "success": false,
  "message": "Resource not found"
}
```

65. CRUD Deep Dive

CRUD means:

Create
Read
Update
Delete

Typical HTTP mapping:

CRUD	HTTP
Create	POST
Read	GET
Update	PUT/PATCH
Delete	DELETE

Create

POST /users

Success may return:

201 Created

Invalid input may return:

400 Bad Request

Read

Get all:

GET /users

Get one:

GET /users/123

Update

Replace:

PUT /users/123

Partial update:

PATCH /users/123

Delete

DELETE /users/123

66. Pagination

Returning thousands/millions of records in one response is inefficient.

Instead:

```
GET /users?page=2&limit=20
```

Pagination reduces:

- Response size
 - Database load
 - Network usage
 - Client processing
-

67. Search API

Example:

```
GET /users?search=sourabh
```

Search allows clients to find relevant resources.

68. Sorting

Example:

```
GET /users?sort=name
```

or:

```
GET /users?sort=-createdAt
```

The exact syntax depends on the API design.

69. Filtering

Example:

```
GET /products?category=laptop&priceMax=100000
```

Filtering restricts results based on criteria.

70. CRUD API Best Practices

Important practices:

- Strict input validation
 - Consistent response formatting
 - Pagination
 - Authentication
 - Authorization
 - Proper error handling
 - Sensitive-field redaction
 - Reasonable payload limits
-

71. RESTful Architecture

REST encourages designing APIs around **resources** and using HTTP semantics properly.

Example:

Good:

```
GET    /users
POST   /users
GET    /users/123
PATCH /users/123
DELETE /users/123
```

Rather than designing everything around action-like URLs.

The key idea is:

Design APIs around resources and use HTTP methods according to their semantics.

72. REST API Versioning

Common approaches:

URI

```
/api/v1/users
```

Query String

```
/api/users?version=1
```

Header

Version specified through headers.

Media Type

Version specified through content negotiation/media type.

73. OpenAPI-Aware API Design

APIs should be designed with clear contracts.

An API contract should describe:

- Endpoints
- Parameters
- Request bodies
- Responses
- Schemas
- Security requirements

This makes APIs easier to consume and document.

74. Large Requests and Responses

Large payloads can cause:

- High memory usage
- High network cost
- Slow responses

Possible solutions include:

- Pagination
 - Compression
 - Selecting only required fields
 - Streaming where appropriate
 - Payload limits
-

75. Databases

Databases are a fundamental part of backend engineering.

Two major categories:

Relational databases
Non-relational databases

76. Relational Databases

Data is generally organized into tables with relationships.

Examples conceptually:

Users
Orders
Products

Relationships can be represented using keys.

77. Non-Relational Databases

Often called NoSQL databases.

They can use different models such as document-oriented or key-value approaches.

The choice depends on the application's requirements.

78. Database Theory

Important concepts to understand:

ACID

Properties associated with reliable transactions:

Atomicity
Consistency
Isolation
Durability

CAP Theorem

A distributed-system concept involving trade-offs among:

Consistency
Availability
Partition tolerance

These are foundational concepts for understanding distributed databases and systems.

79. Database Queries and Joins

Backend applications need to:

- Query data
- Insert data
- Update data
- Delete data

- Join related data

Example relationship:

Users
↓
Orders

A join can combine related records.

80. Database Schema Design

Good schema design affects:

- Performance
 - Data integrity
 - Maintainability
 - Query efficiency
-

81. Database Indexing

Indexes can make frequently used queries significantly faster.

For example, if users are frequently searched by email, an index on email can help.

But indexes are not free.

They:

- Consume storage
- Increase write/update overhead

Therefore, index based on actual query patterns.

82. Database Optimization

Important techniques include:

- Query optimization
- Indexing

- Caching
 - Connection pooling
-

83. Data Integrity

Important tools include:

- Constraints
- Validation
- Transactions
- Concurrency control

The goal is to prevent invalid or inconsistent data.

84. ORM

ORM = **Object-Relational Mapping**

It allows developers to interact with relational databases through programming-language objects/models.

Benefits:

- Productivity
- Abstraction
- Less repetitive SQL

Trade-offs:

- Less direct control
- Abstraction can hide inefficient queries
- Complex queries may still require raw SQL

Don't blindly use an ORM without understanding the database underneath.

85. Database Migrations

Migrations track changes to database structure.

Example:

```
Version 1
Users table

    ↓ migration

Version 2
Users + email column

    ↓ migration

Version 3
Users + email index
```

They allow database schema changes to be managed systematically.

86. Business Logic Layer (BLL)

The Business Logic Layer contains the application's **core business rules**.

A simplified architecture:

```
Presentation Layer
    ↓
Business Logic Layer
    ↓
Data Access Layer
    ↓
Database
```

87. Presentation Layer

This layer handles communication with the user/client.

It includes things such as:

- Routing
- Validation
- Middleware
- Handlers
- Controllers

Its responsibility is mainly receiving and returning data.

88. Business Logic Layer

This contains rules that define how the application actually works.

Example:

For a car rental system:

```
Check car availability
    ↓
Calculate rental cost
    ↓
Apply discount rules
    ↓
Create booking
```

These are business rules.

89. Data Access Layer

The data access layer interacts with databases.

Responsibilities include:

- Queries
- Inserts
- Updates
- Deletes

The business logic layer uses this layer to access persistent data.

90. Separation of Concerns

Different components should have different responsibilities.

Important design principles mentioned include:

- Separation of concerns
- Single Responsibility Principle
- Open/Closed Principle
- Dependency Inversion

The goal is to avoid tightly coupled, difficult-to-maintain code.

91. Business Logic Components

Common components include:

Services

Contain application/business operations.

Domain Models

Represent important entities.

Examples:

User
Order
Booking
Payment

Business Rules

Define what operations are allowed.

Business Validation

Checks rules specific to the domain.

92. Error Propagation

If a service encounters an error:

```
Database
↓
Data Access Layer
↓
Service
↓
Controller
↓
HTTP Response
```

Errors should be propagated and translated appropriately rather than randomly handled at every layer.

93. Caching

Caching stores frequently accessed data in a faster location so that it can be reused.

Basic idea:

```
Request
↓
Cache
↓
Cache hit → return data

Cache miss
↓
Database
↓
Store result in cache
↓
Return data
```

Caching improves performance but introduces an important problem:

Keeping cached data consistent with the actual source of truth.

94. Cache vs Database

Database:

Persistent source of truth

Cache:

Fast temporary copy

A cache should generally not be treated as the permanent source of truth.

95. Types of Caching

Examples:

- Browser caching
 - Client-side caching
 - Server-side caching
 - Database caching
 - In-memory caching
 - Distributed/network caching
-

96. Cache Strategies

Important strategies include:

Cache Aside

Application checks cache first.

```
App
↓
Cache
↓ miss
Database
↓
Cache
```

Read Through

The cache layer handles loading missing data.

Write Through

Writes are written through the cache to the underlying storage.

Write Back

Writes may initially go to cache and later be persisted.

97. Cache Eviction Strategies

When cache capacity is limited, old entries must be removed.

Common strategies:

LRU

Least Recently Used

Remove the item that hasn't been used recently.

LFU

Least Frequently Used

Remove the item used least often.

TTL

Time To Live

Remove data after a specified amount of time.

FIFO

First In, First Out

Remove the oldest inserted item.

98. Cache Invalidation

One of the difficult caching problems is:

When should cached data be removed or updated?

Strategies include:

- Manual invalidation
- TTL-based invalidation
- Event-based invalidation

A stale cache can return outdated information.

99. Cache Levels

Level 1

Fast, small, often in-memory cache.

Level 2

Larger distributed/network cache.

Hierarchical Caching

Combines multiple levels:

```
Request
↓
L1 Cache
↓ miss
L2 Cache
↓ miss
Database
```

Frequently accessed data can stay in the fast L1 cache while less frequently accessed data remains in the larger L2 cache.

100. Web Application Caching

Possible cached items:

- Static assets
- API responses
- Database query results

HTTP headers can also be used for browser/client caching.

101. Database Query Caching

Expensive query results can sometimes be cached.

Example:

```
Expensive database query
      ↓
Cache result
      ↓
Future requests use cached result
```

Redis is one technology commonly used for such caching.

102. Cache Hit and Cache Miss

Cache Hit

Requested data exists in cache.

```
Request
  ↓
Cache
  ↓
Found
```

Cache Miss

Requested data isn't in cache.

```
Request
  ↓
Cache
  ↓
Not found
  ↓
Database
```

Important metric:

Cache Hit Ratio

Higher hit ratios generally mean more requests are served from cache.

103. Transactional Emails

Transactional emails are emails triggered by application events.

Examples:

- Account verification
 - Password reset
 - Payment confirmation
 - Notifications
-

104. Anatomy of a Transactional Email

Typical components:

Subject
Preheader
Header
Main content
CTA
Footer

Dynamic information can be inserted to personalize the message.

Example:

Hello {{name}}

105. Task Queuing and Scheduling

Some work should not block the user's request.

Example:

```
graph TD; A[User requests account deletion] --> B[API responds quickly]; B --> C[Background job starts]; C --> D[Delete data from multiple tables];
```

Instead of keeping the HTTP request open while expensive work executes, the work can be delegated to a task queue.

106. Common Task Queue Use Cases

Queues can be used for:

- Sending emails
 - Image processing
 - Payment processing
 - Webhook processing
 - Third-party API calls
 - Heavy computation
 - Data processing
-

107. Scheduling

Scheduled jobs can perform tasks such as:

- Database backups
 - Recurring notifications
 - Reminders
 - Data synchronization
 - Clearing logs
 - Clearing caches
 - Maintenance operations
-

108. Components of a Task Queue

Basic architecture:

```
Producer
↓
Queue / Broker
↓
Consumer / Worker
↓
Task Processing
```

The producer creates a task.

The broker/queue stores and manages it.

The consumer/worker processes it.

109. Task Dependencies

Tasks may depend on one another.

Examples:

Chain

```
Task A
↓
Task B
↓
Task C
```

Parent-child

```
Parent Task
├─ Child 1
├─ Child 2
└─ Child 3
```

110. Concurrent Task Groups

Multiple tasks can sometimes execute concurrently.

Example:

```
Task A ┌
Task B ┤ → Wait for completion
Task C └
```

This can reduce total execution time when tasks are independent.

111. Task Errors and Retries

Background tasks can fail.

Systems should define:

- Error handling
- Retry policies
- Maximum retry attempts

Be careful with retries because repeatedly retrying a permanently failing task can overload the system.

112. Task Prioritization

Not all tasks have equal importance.

Example:

```
Payment Processing
      ↓
Higher priority

Notification Email
      ↓
Lower priority
```

Priority queues can ensure important tasks are processed first.

Rate limiting can also be applied to tasks.

113. Elasticsearch

Elasticsearch is used for powerful search and analytics use cases.

Possible applications:

- Full-text search
 - Type-ahead/autocomplete
 - Log analytics
 - Social-media search
-

114. Elasticsearch Internals

Important concepts include:

Inverted Index

A data structure optimized for text search.

Conceptually:

```
"backend" → document 1, document 5  
"database" → document 2, document 5
```

Instead of scanning every document for every query, the index maps terms to documents.

TF-IDF

Term Frequency – Inverse Document Frequency

A concept used to estimate how important a term is within documents.

Segments

Elasticsearch stores indexed data in segments.

Shards

Indexes can be divided across shards to distribute data and workload.

115. Elasticsearch Search

Search types include:

- Basic search
 - Full-text search
 - Relevance scoring
 - Filtering
 - Aggregation
 - Fuzzy search
-

116. Elasticsearch Optimization

Important ideas:

- Explicit field mappings
 - Choosing appropriate field types
 - Understanding analyzers
 - Boosting relevant fields
 - Efficient pagination
 - Indexing data in batches
 - Avoiding unnecessary wildcards
 - Carefully choosing shard counts
-

117. Kibana

Kibana provides a user-friendly interface for working with Elasticsearch data, particularly for search, visualization, and analytics.

118. Error Handling

Errors commonly fall into categories such as:

Syntax Errors

Problems with program syntax.

Runtime Errors

Problems occurring while the program executes.

Logical Errors

Program runs but produces incorrect results.

119. Error Handling Strategies

Fail Fast

Detect a problem early and stop the operation.

Fail Safe

Prevent failures from causing catastrophic system behavior.

Graceful Degradation

If one feature fails, keep the rest of the system functioning where possible.

Prevention

Design systems so common errors are less likely to occur.

120. Error Handling Best Practices

- Catch errors where appropriate.
 - Don't swallow errors.
 - Create meaningful/custom error types.
 - Log important errors.
 - Preserve useful stack traces.
 - Use centralized error handling.
 - Return safe user-facing messages.
-

121. User-Facing Errors

Don't expose internal implementation details.

Bad:

```
MongoDB connection failed at /src/controllers/user.js line 84...
```

Better:

```
Something went wrong. Please try again later.
```

The detailed error should go to logs, not necessarily to the user.

122. Monitoring Errors

Error monitoring can use tools such as:

- Sentry
- ELK stack

Alerts can be delivered through channels such as:

- Email
- Slack

The important principle is:

An error that nobody notices is still a production problem.

123. Configuration Management

Configuration management separates environment-specific settings from application code.

Examples:

```
Database URL  
API keys  
Passwords  
Certificates  
Feature flags  
Rate limits
```

Instead of hardcoding them:

```
const password = "mySecret123";
```

configuration should be supplied through appropriate configuration mechanisms.

124. Types of Configuration

Static Configuration

Examples:

- Database credentials
- API endpoints

Dynamic Configuration

Examples:

- Feature flags
- Rate limits

Sensitive Configuration

Examples:

- Passwords
- Tokens
- API keys
- Secrets
- Private certificates

125. Sources of Configuration

Possible sources include:

```
Environment variables
JSON
YAML
Command-line flags
Static configuration files
```

Different environments can have different values without changing application logic.

126. Logging, Monitoring and Observability

These concepts are related but different.

Logging

Records events.

Monitoring

Tracks system health and predefined metrics.

Tracing

Tracks the journey of a request through different components.

Observability

The broader ability to understand internal system behavior from its external outputs.

127. Types of Logs

Examples:

- System logs
 - Application logs
 - Access logs
 - Security logs
-

128. Log Levels

Common levels:

```
DEBUG
INFO
WARN
ERROR
FATAL
```

The exact naming can vary between systems.

129. Structured vs Unstructured Logging

Unstructured

User Sourabh logged in at 10:30

Structured

```
{  
  "event": "login",  
  "userId": "123",  
  "timestamp": "..."  
}
```

Structured logs are easier for machines to search, filter, and analyze.

130. Logging Best Practices

Use:

- Centralized logging
- Log rotation
- Appropriate retention
- Meaningful context
- Request IDs/trace IDs where useful

Never casually log sensitive data such as:

Passwords
API keys
Tokens

131. Monitoring

Monitoring can include:

Infrastructure Monitoring

Tracks:

- CPU
- Memory
- Disk
- Network

Application Performance Monitoring

Tracks application behavior and performance.

Uptime Monitoring

Checks whether services are available.

Tools mentioned include:

Prometheus
Grafana

132. Alerts

Alerts should have meaningful thresholds.

Example:

CPU > 90% for 10 minutes

But don't create alerts for everything.

Too many useless alerts create:

Alert fatigue

Good alerts should be actionable.

133. Three Pillars of Observability

The three pillars are:

```
Logs
Metrics
Traces
```

Together they help answer:

```
What happened?
How much/how often?
Where did the request fail or slow down?
```

134. Graceful Shutdown

A server shouldn't simply terminate immediately when shutting down.

Graceful shutdown allows the application to finish ongoing work safely.

Useful during:

- Server restarts
- Cloud scaling
- Microservice deployments
- Long-running jobs

135. Graceful Shutdown Flow

Typical sequence:

```
Shutdown signal
  ↓
Capture signal
  ↓
Stop accepting new requests
  ↓
Complete in-flight requests
```

```
↓  
Close database connections  
↓  
Close files/resources  
↓  
Terminate application
```

Signals mentioned include:

```
SIGINT  
SIGTERM
```

136. Backend Security

Security must be considered throughout the backend.

Important attacks/risks include:

- SQL Injection
- NoSQL Injection
- XSS
- CSRF
- Broken authentication
- Insecure deserialization

137. Secure Software Principles

Least Privilege

Give users/services only the permissions they actually need.

Defense in Depth

Use multiple security layers.

Fail Secure

When something goes wrong, default toward a secure state.

Secure Defaults

Default configuration should be safe.

Separation of Duties

Sensitive operations should not unnecessarily depend on a single unchecked authority.

Security by Design

Security should be considered during system design, not added as an afterthought.

138. Security Practices

Important practices include:

- Input validation
 - Input sanitization
 - Rate limiting
 - Content Security Policy
 - CORS configuration
 - Secure cookies
 - SameSite protections
 - Security monitoring
-

139. Scaling and Performance

Performance is not just about making code execute faster.

Important metrics include:

- Response time
- Resource utilization
- Throughput
- Database performance

The goal is to identify bottlenecks and remove them.

140. Database Performance

Important techniques:

- Proper indexes
 - Query optimization
 - Appropriate joins
 - Avoiding N+1 query problems
 - Lazy loading where appropriate
 - Batch processing
-

141. N+1 Query Problem

A common database performance problem.

Conceptually:

```
1 query → get 100 users
```

Then:

```
100 additional queries → get data for each user
```

Total:

```
1 + 100 = 101 queries
```

This can be much slower than fetching related data efficiently.

142. Batch Processing

Instead of processing huge amounts of data one item at a time, process it in batches.

Example:

```
1,000,000 records
```

Instead of:

```
1 record × 1,000,000 operations
```

Use:
10,000 records × 100 batches

This can reduce memory usage and database overhead.

143. Memory Leaks

Long-running backend applications must properly release resources.

Potential resources include:

- File handles
- Database connections
- Network connections
- Large objects

Failure to clean them up can cause memory/resource exhaustion.

144. Network Optimization

Reduce network overhead by:

- Reducing payload size
 - Removing unnecessary fields
 - Using compression
 - Caching where appropriate
-

145. Performance Testing and Profiling

Before optimizing, identify the actual bottleneck.

Use:

- Performance testing
- Profiling
- Metrics

Don't optimize random parts of the application based on assumptions.

146. Avoid Premature Optimization

First prioritize:

```
Correctness
↓
Clarity
↓
Maintainability
↓
Measure performance
↓
Optimize actual bottlenecks
```

Writing complicated code "because it might be faster" without measurements is usually bad engineering.

147. Modular Code

Modular systems make optimization easier.

For example:

```
Authentication Module
Database Module
Payment Module
Search Module
Notification Module
```

Individual components can be improved without rewriting the entire system.

148. Graceful Degradation

If a non-critical component becomes unavailable, the entire application should not necessarily crash.

Example:

```
Main API → working
Notification service → unavailable
```

The application may still complete the main operation and process the notification later.

149. Offloading Non-Critical Tasks

Move expensive/non-critical tasks into background processing.

Examples:

- Sending emails
- Logging
- Image processing
- Notifications

This keeps critical request paths fast.

150. Concurrency vs Parallelism

These concepts are different.

Concurrency

Multiple tasks make progress during overlapping periods.

Useful especially for:

I/O-bound work

Example:

```
Waiting for database
Waiting for API
Waiting for file
```

The system can make progress on other work while waiting.

Parallelism

Multiple tasks actually execute at the same time using multiple processing resources.

Useful especially for:

CPU-bound work

Example:

Heavy computation
Image processing
Large calculations

Remember

I/O-bound → concurrency is highly useful
CPU-bound → parallelism can be highly useful

151. Object Storage and Large Files

Object storage is commonly used for large files.

Example:

Amazon S3

Instead of storing huge files directly inside the database, applications often store them in object storage and keep metadata/URLs in the database.

152. Handling Large Files

Important techniques:

Chunking

Split a large file into smaller pieces.

Large File
↓
Chunk 1

```
Chunk 2  
Chunk 3  
...
```

Streaming

Process data progressively instead of loading the entire file into memory.

Multipart Upload

Upload a large file as multiple parts.

These approaches improve reliability and memory usage for large files.

153. Real-Time Backend Systems

Some applications require communication in real time.

Examples:

- Chat
- Live notifications
- Collaborative applications

Technologies/concepts include:

```
WebSockets  
Server-Sent Events  
Events  
Pub/Sub architecture
```

154. Testing and Code Quality

Backend systems require multiple types of testing.

Unit Testing

Tests individual components/functions.

Integration Testing

Tests interactions between components.

Example:

API + Database

End-to-End Testing

Tests complete user/system flows.

Functional Testing

Checks whether required functionality works.

Regression Testing

Ensures existing functionality does not break after changes.

Performance Testing

Measures system performance.

Load Testing

Tests behavior under expected/high load.

Stress Testing

Pushes the system beyond normal capacity to identify breaking points.

User Acceptance Testing

Checks whether the system meets user/business requirements.

Security Testing

Identifies security vulnerabilities.

155. Test-Driven Development

TDD = **Test-Driven Development**

General cycle:

```
Write test
↓
Implement code
↓
Make test pass
↓
Refactor
```

The transcript also emphasizes automating tests in CI/CD environments.

156. Code Quality

Code quality can be supported using:

- Linters
 - Formatters
 - Automated tests
 - Code review
 - CI/CD checks
-

157. Code Coverage

Coverage measures how much of the code is exercised by tests.

Higher coverage does **not automatically mean better software**.

Bad tests can produce high coverage while failing to test important behavior.

158. Cyclomatic Complexity

Cyclomatic complexity measures the complexity of code based on the number of possible paths through it.

More branches generally mean:

```
More possible paths
↓
```

Higher complexity



Harder testing/maintenance

159. Maintainability Index

Maintainability index attempts to quantify how easy code is to maintain.

It considers factors related to things such as:

- Complexity
- Lines of code
- Other code characteristics

The goal is to identify code that may be difficult to maintain.

160. 12-Factor App

The **12-Factor App** is a set of principles for building applications suitable for modern deployment environments.

It is an important topic for backend engineers, particularly for applications designed to run consistently across environments and scale.

The transcript introduces it as a dedicated topic to study further.

161. OpenAPI Standards

OpenAPI is a standard for describing APIs.

It can help with:

- API documentation
 - Automation
 - Client generation
 - Testing
 - Tooling ecosystem
-

162. Swagger and OpenAPI

Swagger was historically associated with the API specification and tooling ecosystem.

The specification evolved into the **OpenAPI Specification**.

Tools such as:

```
Swagger UI  
Postman
```

can work with API definitions and documentation.

163. OpenAPI Document Structure

An OpenAPI document can describe:

Metadata

Information about the API.

Paths

Available endpoints.

Example:

```
/users  
/users/{id}
```

Parameters

Inputs accepted by endpoints.

Request/Response Definitions

What clients send and what servers return.

Schemas

Structure and types of data.

Components

Reusable definitions.

Security Definitions

Authentication/security mechanisms.

Responses

Expected API responses.

164. OpenAPI Versions

Important versions mentioned:

```
OpenAPI 3.0
OpenAPI 3.1
```

These specifications provide modern ways to describe APIs.

165. API-First Development

Traditional approach:

```
Build API
↓
Document it
```

API-first:

```
Define API contract/specification
↓
Design API
↓
Implement API
```

This helps teams agree on the API contract before implementation.

166. Webhooks

A webhook allows one system to notify another system when an event occurs.

Basic flow:

```
Event occurs
↓
Server sends HTTP request
↓
Webhook URL
↓
Receiver processes event
```

167. API vs Webhook

API polling

Client asks repeatedly:

```
Client → "Anything new?"
Server → "No"

Client → "Anything new?"
Server → "No"

Client → "Anything new?"
Server → "Yes"
```

Webhook

The server pushes the event:

```
Event occurs
↓
Server → Client
```

The important difference:

Polling → client initiated
Webhook → server/event initiated

168. Webhook Components

A webhook typically involves:

- Webhook URL
- Event trigger
- Payload
- HTTP method
- Response handling

Example:

```
Payment successful
    ↓
Payment provider
    ↓
POST /webhooks/payment
    ↓
Your server
```

169. Webhook Security

Important practices:

- Signature verification
- HTTPS
- Fast response
- Retry handling
- Logging

Signature verification helps ensure the webhook actually came from the expected provider.

170. Webhook Real-World Use Cases

Examples include:

- Payment processing
 - GitHub events
 - Slack events
 - Discord events
 - Third-party integrations
-

171. DevOps for Backend Engineers

Backend engineers should understand core DevOps concepts.

Important areas:

```
CI
CD
Infrastructure as Code
Configuration Management
Version Control
Containers
Orchestration
Deployment
Scaling
```

172. CI/CD

Continuous Integration

Frequently integrate code changes and automatically test them.

Continuous Delivery

Keep software in a state where it can be released reliably.

Continuous Deployment

Automatically deploy validated changes to production.

173. Containers

Docker allows applications and their dependencies to be packaged into containers.

Basic idea:

```
Application
+
Dependencies
+
Configuration
  ↓
Container
```

Containers make environments more consistent.

174. Kubernetes

Kubernetes is used for container orchestration.

It can help manage:

- Containerized applications
 - Scaling
 - Service deployment
 - Availability
 - Workloads
-

175. Infrastructure as Code

Infrastructure configuration can be represented and managed as code.

Benefits include:

- Reproducibility
 - Version control
 - Automation
 - Consistency
-

176. Scaling

Two basic scaling approaches:

Vertical Scaling

Increase resources of one machine.

```
More CPU
More RAM
```

Horizontal Scaling

Add more machines/instances.

```
Server 1
Server 2
Server 3
```

Horizontal scaling is particularly important when building distributed systems.

177. Deployment Strategies

Different deployment strategies can reduce downtime and deployment risk.

Examples include:

Blue-Green / Red-Green Deployment

Maintain two environments and switch traffic between them.

Rolling Deployment

Gradually replace old instances with new instances.

Conceptually:

```
Old version
Old version
Old version
```

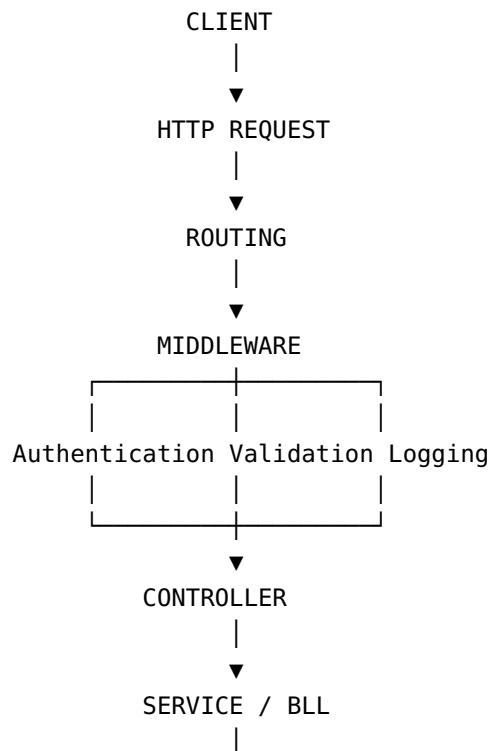
↓
New version
Old version
Old version

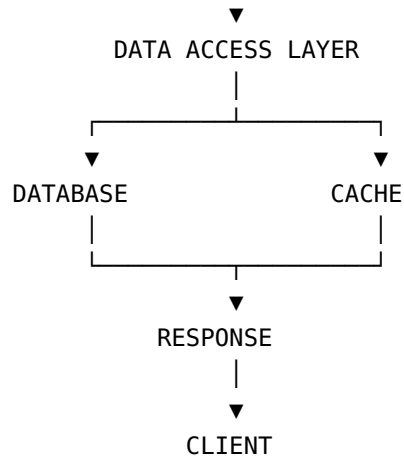
↓
New version
New version
Old version

↓
New version
New version
New version

178. Complete Backend Engineering Mental Model

Keep this high-level flow in your head:



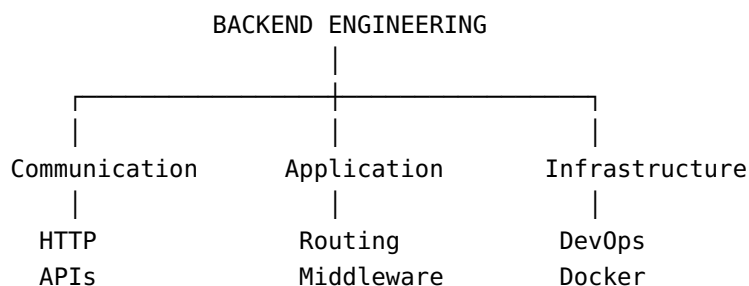


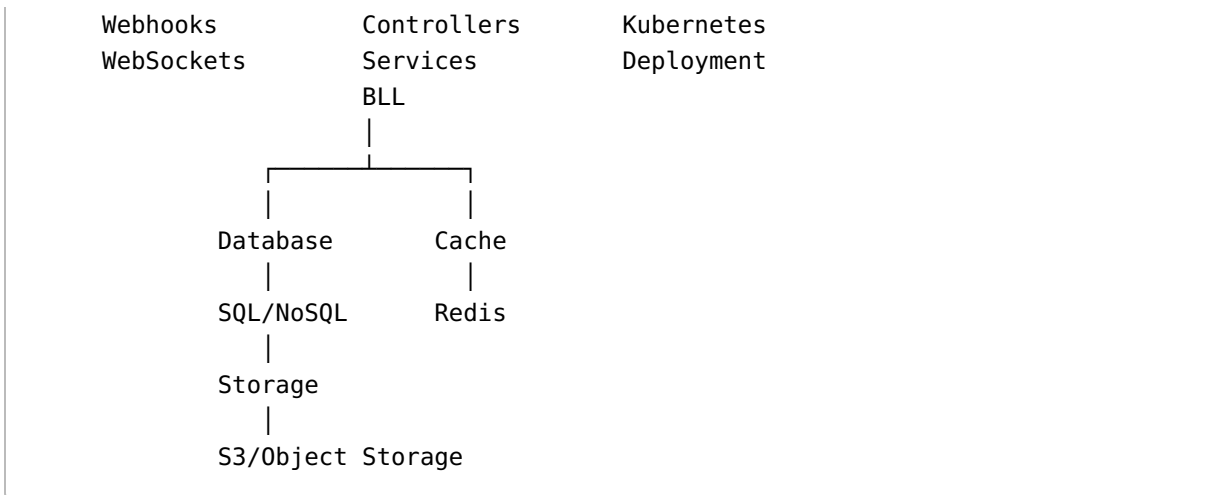
Around this core flow, additional infrastructure exists:

- Authentication
- Authorization
- Caching
- Queues
- Search
- Logging
- Monitoring
- Tracing
- Security
- Testing
- Object Storage
- Real-time communication
- DevOps
- Scaling

179. The Big Picture of Backend Engineering

Backend engineering can be thought of as several interconnected layers:





And across every layer:

Security
Performance
Reliability
Observability
Testing
Maintainability
Scalability

180. Most Important Mental Connections

Connection 1 — HTTP → Routing

HTTP Request
↓
Method + URL
↓
Router
↓
Correct handler

Connection 2 — Middleware → Request Context

Middleware
↓
Authenticate user

```
↓
Attach user/request information
↓
Controller/Service
```

Connection 3 — Controller → Business Logic → Database

```
Controller
↓
Service
↓
Data Access Layer
↓
Database
```

Connection 4 — Cache → Database

```
Request
↓
Cache
↓ hit → return
↓ miss
Database
↓
Cache
↓
Return
```

Connection 5 — Heavy Work → Queue

```
Client
↓
API
↓
Quick response
↓
Queue
↓
Worker
↓
Heavy processing
```

Connection 6 — Production System → Observability

```
Application
├─ Logs
├─ Metrics
└─ Traces
    ↓
Observability
    ↓
Understand failures/performance
```

Connection 7 — Traffic Growth → Scaling

```
More traffic
    ↓
Performance bottleneck
    ↓
Identify bottleneck
    ↓
Optimize
    ↓
If necessary → Scale
```

181. Day 1 — What You Should Remember

Do **not** try to memorize every definition today.

The main purpose of Day 1 is to build the map of backend engineering.

You should understand these major blocks:

1. Backend Engineering Fundamentals
2. HTTP
3. Routing
4. Serialization / Deserialization
5. Authentication / Authorization
6. Validation / Transformation
7. Middleware
8. Request Context
9. Controllers / Services
10. CRUD
11. REST

12. Databases
13. Business Logic Layer
14. Caching
15. Transactional Emails
16. Queues / Scheduling
17. Elasticsearch
18. Error Handling
19. Configuration
20. Logging / Monitoring / Observability
21. Graceful Shutdown
22. Security
23. Scaling / Performance
24. Concurrency / Parallelism
25. Object Storage
26. Real-Time Systems
27. Testing / Code Quality
28. 12-Factor App
29. OpenAPI
30. Webhooks
31. DevOps

The most important mental model is:

A backend is not an Express server with some routes and MongoDB attached. It is a complete system responsible for communication, data, business rules, security, reliability, performance, observability, and deployment.

That distinction is what separates "**I can build APIs**" from **backend engineering**.

Source transcript: